

VELORA — Module POS (Part 2)

Repositories · DTOs · Form Requests · Resources

4. Repository Interfaces

app/Modules/POS/Repositories/Contracts/CategoryRepositoryInterface.php

```
<?php
```

```
namespace App\Modules\POS\Repositories\Contracts;
```

```
use App\Modules\POS\Models\Category;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
interface CategoryRepositoryInterface
```

```
{  
    public function findById(string $id): ?Category;  
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator;  
    public function allActive(string $businessId): Collection;  
    public function getTree(string $businessId): Collection;  
    public function create(array $data): Category;  
    public function update(string $id, array $data): Category;  
    public function delete(string $id): bool;  
}
```

app/Modules/POS/Repositories/Contracts/UnitRepositoryInterface.php

```
<?php
```

```
namespace App\Modules\POS\Repositories\Contracts;
```

```
use App\Modules\POS\Models\Unit;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
interface UnitRepositoryInterface
```

```
{
```

```
    public function findById(string $id): ?Unit;
```

```
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator;
```

```
    public function allActive(string $businessId): Collection;
```

```
    public function create(array $data): Unit;
```

```
    public function update(string $id, array $data): Unit;
```

```
    public function delete(string $id): bool;
```

```
}
```

app/Modules/POS/Repositories/Contracts/ProductRepositoryInterface.php

```
<?php
```

```
namespace App\Modules\POS\Repositories\Contracts;
```

```
use App\Modules\POS\Models\Product;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
interface ProductRepositoryInterface
```

```
{
```

```
    public function findById(string $id): ?Product;
```

```
    public function findBySku(string $businessId, string $sku): ?Product;
```

```
    public function findByBarcode(string $businessId, string $barcode): ?Product;
```

```
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator;
```

```
    public function search(string $businessId, string $query): LengthAwarePaginator;
```

```
    public function countByBusiness(string $businessId): int;
```

```
    public function create(array $data): Product;
```

```
    public function update(string $id, array $data): Product;
```

```
    public function delete(string $id): bool;
```

```
    // Variants
```

```
    public function createVariant(array $data): \App\Modules\POS\Models\ProductVariant;
```

```
    public function updateVariant(string $variantId, array $data): \App\Modules\POS\Models\ProductVariant;
```

```
    public function deleteVariant(string $variantId): bool;
```

```
}
```

5. Repository Implementations

app/Modules/POS/Repositories/Eloquent/CategoryRepository.php

```
<?php

namespace App\Modules\POS\Repositories\Eloquent;

use App\Modules\POS\Models\Category;
use App\Modules\POS\Repositories\Contracts\CategoryRepositoryInterface;
use App\Shared\Repositories\BaseRepository;
use Illuminate\Database\Eloquent\Collection;
use Illuminate\Pagination\LengthAwarePaginator;

class CategoryRepository extends BaseRepository implements CategoryRepositoryInterface
{
    public function __construct(Category $model)
    {
        parent::__construct($model);
    }

    public function findById(string $id): ?Category
    {
        return Category::with('parent')->find($id);
    }

    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator
    {
        return Category::where('business_id', $businessId)
            ->when($filters['search'] ?? null, fn($q, $v) => $q->where('name', 'like', "%{$v}%"))
            ->when(isset($filters['is_active']), fn($q) => $q->where('is_active', $filters['is_active']))
            ->with('parent')
            ->orderBy('sort_order')
            ->latest()
            ->paginate($filters['per_page'] ?? 15);
    }

    public function allActive(string $businessId): Collection
    {
        return Category::where('business_id', $businessId)
            ->active()
            ->get();
    }

    public function getTree(string $businessId): Collection
    {

```

```

        return Category::where('business_id', $businessId)
            ->active()
            ->root()
            ->with(['children' => fn($q) => $q->active()->orderBy('sort_order')])
            ->orderBy('sort_order')
            ->get();
    }

    public function create(array $data): Category
    {
        return Category::create($data);
    }

    public function update(string $id, array $data): Category
    {
        $category = Category::findOrFail($id);
        $category->update($data);
        return $category->fresh();
    }

    public function delete(string $id): bool
    {
        return Category::findOrFail($id)->delete();
    }
}

```

app/Modules/POS/Repositories/Eloquent/UnitRepository.php

```
<?php
```

```
namespace App\Modules\POS\Repositories\Eloquent;
```

```
use App\Modules\POS\Models\Unit;
```

```
use App\Modules\POS\Repositories\Contracts\UnitRepositoryInterface;
```

```
use App\Shared\Repositories\BaseRepository;
```

```
use Illuminate\Database\Eloquent\Collection;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
class UnitRepository extends BaseRepository implements UnitRepositoryInterface
```

```
{
```

```
    public function __construct(Unit $model)
```

```
    {
```

```
        parent::__construct($model);
```

```
    }
```

```
    public function findById(string $id): ?Unit
```

```
    {
```

```
        return Unit::find($id);
```

```
    }
```

```
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator
```

```
    {
```

```
        return Unit::where('business_id', $businessId)
```

```
            ->when($filters['search'] ?? null, fn($q, $v) =>
```

```
                $q->where(fn($q) => $q->where('name', 'like', "%{$v}%")->orWhere('symbol', 'like', "%{$v}%")
```

```
            )
```

```
            ->latest()
```

```
            ->paginate($filters['per_page'] ?? 15);
```

```
    }
```

```
    public function allActive(string $businessId): Collection
```

```
    {
```

```
        return Unit::where('business_id', $businessId)
```

```
            ->active()
```

```
            ->orderBy('name')
```

```
            ->get();
```

```
    }
```

```
    public function create(array $data): Unit
```

```
    {
```

```
        return Unit::create($data);
```

```
    }
```

```
public function update(string $id, array $data): Unit
{
    $unit = Unit::findOrFail($id);
    $unit->update($data);
    return $unit->fresh();
}

public function delete(string $id): bool
{
    return Unit::findOrFail($id)->delete();
}
}
```

app/Modules/POS/Repositories/Eloquent/ProductRepository.php

```
<?php
```

```
namespace App\Modules\POS\Repositories\Eloquent;
```

```
use App\Modules\POS\Models\Product;
```

```
use App\Modules\POS\Models\ProductVariant;
```

```
use App\Modules\POS\Repositories\Contracts\ProductRepositoryInterface;
```

```
use App\Shared\Repositories\BaseRepository;
```

```
use Illuminate\Pagination\LengthAwarePaginator;
```

```
class ProductRepository extends BaseRepository implements ProductRepositoryInterface
```

```
{
```

```
    public function __construct(Product $model)
```

```
    {
```

```
        parent::__construct($model);
```

```
    }
```

```
    public function findById(string $id): ?Product
```

```
    {
```

```
        return Product::with(['category', 'unit', 'variants', 'images'])->find($id);
```

```
    }
```

```
    public function findBySku(string $businessId, string $sku): ?Product
```

```
    {
```

```
        return Product::where('business_id', $businessId)->where('sku', $sku)->first();
```

```
    }
```

```
    public function findByBarcode(string $businessId, string $barcode): ?Product
```

```
    {
```

```
        return Product::where('business_id', $businessId)->where('barcode', $barcode)->first();
```

```
    }
```

```
    public function findByBusiness(string $businessId, array $filters = []): LengthAwarePaginator
```

```
    {
```

```
        return Product::where('business_id', $businessId)
```

```
            ->when($filters['category_id'] ?? null, fn($q, $v) => $q->where('category_id', $v))
```

```
            ->when($filters['status'] ?? null, fn($q, $v) => $q->where('status', $v))
```

```
            ->when($filters['type'] ?? null, fn($q, $v) => $q->where('type', $v))
```

```
            ->when($filters['search'] ?? null, fn($q, $v) =>
```

```
                $q->where(fn($q) => $q->where('name', 'like', "%{$v}%")->orWhere('sku', 'like', "%{$v}%")
```

```
            )
```

```
            ->with(['category', 'unit', 'images' => fn($q) => $q->where('is_primary', true)])
```

```
            ->latest()
```

```
            ->paginate($filters['per_page'] ?? 15);
```

```
    }
```



```

public function search(string $businessId, string $query): LengthAwarePaginator
{
    return Product::where('business_id', $businessId)
        ->active()
        ->where(fn($q) =>
            $q->where('name', 'like', "%{$query}%")
                ->orWhere('sku', 'like', "%{$query}%")
                ->orWhere('barcode', 'like', "%{$query}%")
            )
        ->with(['category', 'unit', 'variants'])
        ->paginate(15);
}

public function countByBusiness(string $businessId): int
{
    return Product::where('business_id', $businessId)->count();
}

public function create(array $data): Product
{
    return Product::create($data);
}

public function update(string $id, array $data): Product
{
    $product = Product::findOrFail($id);
    $product->update($data);
    return $product->fresh();
}

public function delete(string $id): bool
{
    return Product::findOrFail($id)->delete();
}

// Variants
public function createVariant(array $data): ProductVariant
{
    return ProductVariant::create($data);
}

public function updateVariant(string $variantId, array $data): ProductVariant
{
    $variant = ProductVariant::findOrFail($variantId);
    $variant->update($data);
    return $variant->fresh();
}

```

```

    }

    public function deleteVariant(string $variantId): bool
    {
        return ProductVariant::findOrFail($variantId)->delete();
    }
}

```

6. Register Repository Bindings

// Tambahkan ke app/Providers/RepositoryServiceProvider.php

```

use App\Modules\POS\Repositories\Contracts\CategoryRepositoryInterface;
use App\Modules\POS\Repositories\Eloquent\CategoryRepository;
use App\Modules\POS\Repositories\Contracts\UnitRepositoryInterface;
use App\Modules\POS\Repositories\Eloquent\UnitRepository;
use App\Modules\POS\Repositories\Contracts/ProductRepositoryInterface;
use App\Modules\POS\Repositories\Eloquent\ProductRepository;

$this->app->bind(CategoryRepositoryInterface::class, CategoryRepository::class);
$this->app->bind(UnitRepositoryInterface::class, UnitRepository::class);
$this->app->bind(ProductRepositoryInterface::class, ProductRepository::class);

```

7. DTOs

app/Modules/POS/DTOs/CreateProductDTO.php

```
<?php

namespace App\Modules\POS\DTOs;

class CreateProductDTO
{
    public function __construct(
        public readonly string $businessId,
        public readonly string $name,
        public readonly string $type,
        public readonly ?string $categoryId      = null,
        public readonly ?string $unitId          = null,
        public readonly ?string $sku             = null,
        public readonly ?string $barcode         = null,
        public readonly ?string $description     = null,
        public readonly int     $costPrice       = 0,
        public readonly int     $sellingPrice    = 0,
        public readonly int     $minPrice        = 0,
        public readonly int     $taxRate         = 0,
        public readonly bool    $taxInclusive    = false,
        public readonly string  $status          = 'active',
        public readonly bool    $hasVariants     = false,
        public readonly bool    $trackStock      = true,
        public readonly bool    $allowNegativeStock = false,
        public readonly bool    $isFeatured      = false,
        public readonly int     $stockQuantity   = 0,
        public readonly int     $stockAlertQty   = 5,
        public readonly array   $variants        = [],
    ) {}

    public static function fromRequest(array $data, string $businessId): static
    {
        return new static(
            businessId:      $businessId,
            name:            $data['name'],
            type:            $data['type'] ?? 'simple',
            categoryId:      $data['category_id'] ?? null,
            unitId:          $data['unit_id'] ?? null,
            sku:             $data['sku'] ?? null,
            barcode:         $data['barcode'] ?? null,
            description:     $data['description'] ?? null,
            costPrice:       $data['cost_price'] ?? 0,
```

```

        sellingPrice:      $data['selling_price'] ?? 0,
        minPrice:          $data['min_price'] ?? 0,
        taxRate:           $data['tax_rate'] ?? 0,
        taxInclusive:      $data['tax_inclusive'] ?? false,
        status:            $data['status'] ?? 'active',
        hasVariants:       $data['has_variants'] ?? false,
        trackStock:        $data['track_stock'] ?? true,
        allowNegativeStock: $data['allow_negative_stock'] ?? false,
        isFeatured:        $data['is_featured'] ?? false,
        stockQuantity:     $data['stock_quantity'] ?? 0,
        stockAlertQty:     $data['stock_alert_qty'] ?? 5,
        variants:          $data['variants'] ?? [],
    );
}

public function toArray(): array
{
    return [
        'business_id'      => $this->businessId,
        'category_id'      => $this->categoryId,
        'unit_id'          => $this->unitId,
        'name'              => $this->name,
        'sku'              => $this->sku,
        'barcode'           => $this->barcode,
        'description'       => $this->description,
        'cost_price'        => $this->costPrice,
        'selling_price'     => $this->sellingPrice,
        'min_price'         => $this->minPrice,
        'tax_rate'          => $this->taxRate,
        'tax_inclusive'     => $this->taxInclusive,
        'type'              => $this->type,
        'status'            => $this->status,
        'has_variants'      => $this->hasVariants,
        'track_stock'       => $this->trackStock,
        'allow_negative_stock' => $this->allowNegativeStock,
        'is_featured'       => $this->isFeatured,
        'stock_quantity'    => $this->stockQuantity,
        'stock_alert_qty'   => $this->stockAlertQty,
    ];
}
}

```

8. Form Requests

app/Modules/POS/Requests/StoreProductRequest.php

```
<?php

namespace App\Modules\POS\Requests;

use App\Modules\POS\Enums\ProductStatus;
use App\Modules\POS\Enums\ProductType;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Enum;

class StoreProductRequest extends FormRequest
{
    public function authorize(): bool { return true; }

    public function rules(): array
    {
        $businessId = auth()->user()->business_id;

        return [
            'name' => ['required', 'string', 'max:200'],
            'category_id' => ['nullable', 'uuid', Rule::exists('categories', 'id')->where('business_id', $businessId)],
            'unit_id' => ['nullable', 'uuid', Rule::exists('units', 'id')->where('business_id', $businessId)],
            'sku' => ['nullable', 'string', 'max:100', Rule::unique('products')->where('business_id', $businessId)],
            'barcode' => ['nullable', 'string', 'max:100', Rule::unique('products')->where('business_id', $businessId)],
            'description' => ['nullable', 'string'],
            'cost_price' => ['required', 'integer', 'min:0'],
            'selling_price' => ['required', 'integer', 'min:0'],
            'min_price' => ['nullable', 'integer', 'min:0'],
            'tax_rate' => ['nullable', 'integer', 'min:0', 'max:100'],
            'tax_inclusive' => ['nullable', 'boolean'],
            'type' => ['required', new Enum(ProductType::class)],
            'status' => ['required', new Enum(ProductStatus::class)],
            'has_variants' => ['nullable', 'boolean'],
            'track_stock' => ['nullable', 'boolean'],
            'allow_negative_stock' => ['nullable', 'boolean'],
            'is_featured' => ['nullable', 'boolean'],
            'stock_quantity' => ['nullable', 'integer', 'min:0'],
            'stock_alert_qty' => ['nullable', 'integer', 'min:0'],

            // Validation for variants if has_variants is true
            'variants' => ['array', 'required_if:has_variants,true'],
            'variants.*.name' => ['required_with:variants', 'string', 'max:150'],
        ];
    }
}
```

```
'variants.*.sku'           => ['nullable', 'string', 'max:100'],
'variants.*.barcode'       => ['nullable', 'string', 'max:100'],
'variants.*.cost_price'     => ['required_with:variants', 'integer', 'min:0'],
'variants.*.selling_price' => ['required_with:variants', 'integer', 'min:0'],
'variants.*.stock_quantity'=> ['nullable', 'integer', 'min:0'],
'variants.*.attributes'     => ['nullable', 'array'],
];
}
}
```

9. Resources

app/Modules/POS/Resources/ProductResource.php

```
<?php

namespace App\Modules\POS\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class ProductResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'                => $this->id,
            'name'              => $this->name,
            'sku'               => $this->sku,
            'barcode'          => $this->barcode,
            'description'       => $this->description,
            'category'          => new CategoryResource($this->whenLoaded('category')),
            'unit'              => new UnitResource($this->whenLoaded('unit')),

            'pricing' => [
                'cost_price'    => $this->cost_price,
                'selling_price' => $this->selling_price,
                'min_price'     => $this->min_price,
                'tax_rate'      => $this->tax_rate,
                'tax_inclusive' => $this->tax_inclusive,
                'price_with_tax' => $this->getPriceWithTax(),
            ],

            'type'              => $this->type,
            'type_label'        => $this->type->label(),
            'status'            => $this->status,
            'status_label'      => $this->status->label(),
            'is_featured'       => $this->is_featured,

            'inventory' => [
                'has_variants'  => $this->has_variants,
                'track_stock'   => $this->track_stock,
                'allow_negative_stock' => $this->allow_negative_stock,
                'stock_quantity' => $this->stock_quantity,
                'stock_alert_qty' => $this->stock_alert_qty,
                'is_low_stock'  => $this->track_stock && ($this->stock_quantity <= $this->stock_a
```

```

        'in_stock'            => $this->hasEnoughStock(1),
    ],

    'image_url'              => $this->image_url,
    'variants'               => ProductVariantResource::collection($this->whenLoaded('variants')),
    'created_at'             => $this->created_at->toIso8601String(),
];
    }
}

```

app/Modules/POS/Resources/ProductVariantResource.php

```

<?php

namespace App\Modules\POS\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class ProductVariantResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'                => $this->id,
            'product_id'        => $this->product_id,
            'name'              => $this->name,
            'sku'               => $this->sku,
            'barcode'           => $this->barcode,
            'cost_price'        => $this->cost_price,
            'selling_price'     => $this->selling_price,
            'min_price'         => $this->min_price,
            'attributes'        => $this->attributes,
            'stock_quantity'    => $this->stock_quantity,
            'stock_alert_qty'   => $this->stock_alert_qty,
            'is_active'         => $this->is_active,
            'image_url'         => $this->image_url,
            'in_stock'          => $this->hasEnoughStock(1),
        ];
    }
}

```